

Sarthi Platform: Detailed Technical Requirements Supplement

1. AI Workflow Error Handling & Edge Cases

1.1 AI Flow Failure Scenarios

Requirement: All AI flows must implement standardized error handling with specific user feedback patterns.

Error Types & Responses:

```
typescript

interface AIFlowError {
  type: 'timeout' | 'rate_limit' | 'invalid_input' | 'service_unavailable' | 'unknown';
  message: string;
  retryable: boolean;
  fallback?: 'manual_mode' | 'cached_result' | 'simplified_flow';
}
```

Specific Implementation Rules:

1. Timeout (> 30 seconds):

- Show: "AI processing is taking longer than expected. You can continue manually or wait."
- Provide: "Continue Manually" and "Keep Waiting" buttons
- Fallback: Enable manual data entry with pre-filled templates

2. Rate Limiting:

- Show: "AI service is temporarily busy. Please try again in 30 seconds."
- Behavior: Auto-retry after 30 seconds, max 3 attempts
- Fallback: After 3 attempts, switch to manual mode

3. Invalid Input:

- Show: "Unable to process this document. Please check the image quality and try again."
- Provide: Image upload guidelines and retry option
- Log: Specific validation failure reason for debugging

4. Service Unavailable:

- Show: "AI assistant is temporarily unavailable. You can complete this task manually."
- Behavior: Immediately switch to manual mode

- Background: Continue attempting to reconnect

1.2 Loading State Specifications

Standard Loading Pattern:

```
typescript

interface LoadingState {
  isLoading: boolean;
  stage: 'initializing' | 'processing' | 'analyzing' | 'finalizing';
  progress?: number; // 0-100 for deterministic processes
  estimatedTimeRemaining?: number; // seconds
  cancellable: boolean;
}
```

Specific Loading Messages by Component:

- **AI Scribe:** "Listening and transcribing..." → "Generating clinical note..." → "Suggesting billing codes..."
- **Lab Processing:** "Reading document..." → "Extracting lab values..." → "Analyzing results..."
- **Check-in Kiosk:** "Processing insurance card..." → "Verifying patient information..." → "Finalizing check-in..."

Button State Rules:

- Loading buttons must show spinner + text change
- Disabled state during processing: opacity: 0.5, cursor: not-allowed
- Progress indicators required for processes >5 seconds
- Cancel button required for processes >15 seconds

2. LocalStorage Data Management & Synchronization

2.1 Data Schema Definitions

Core Data Models:

```
typescript
```

```

interface CheckedInPatient {
  id: string;
  name: string;
  checkInTime: string; // ISO 8601
  status: 'waiting_for_front_office' | 'waiting_for_ma' | 'roomed' | 'ready_for_doctor' | 'attention_needed' | 'completed';
  roomNumber?: string;
  readyForDoctorTime?: string; // ISO 8601
  maRequestNotes?: string;
  reasonForVisit: string;
  appointmentId?: string;
}

interface Claim {
  id: string;
  patientId: string;
  patientName: string;
  dateOfService: string;
  status: 'Needs Review' | 'Ready to Submit' | 'Submitted' | 'Paid' | 'Denied';
  lineItems: ClaimLineItem[];
  flaggedIssues?: string[];
  denialRiskScore: 'Low' | 'Medium' | 'High';
  denialRiskReasoning?: string;
  suggestionToReduceRisk?: string;
  createdAt: string; // ISO 8601
  lastModified: string; // ISO 8601
}

interface LabResult {
  id: string;
  patientId: string;
  patientName: string;
  testName: string;
  workflowStatus: 'pending_technician' | 'pending_doctor' | 'approved_by_doctor' | 'sent_to_emr';
  results?: LabValue[];
  doctorNotes?: string;
  technicianNotes?: string;
  createdAt: string;
  lastModified: string;
}

```

2.2 Synchronization Rules

Storage Event Handling:

typescript

// Required in every component that reads localStorage

```
useEffect(() => {  
  const handleStorageChange = (e: StorageEvent) => {  
    if (e.key === 'checkedInPatients' && e.newValue) {  
      setPatients(JSON.parse(e.newValue));  
    }  
  };  
  
  window.addEventListener('storage', handleStorageChange);  
  return () => window.removeEventListener('storage', handleStorageChange);  
}, []);
```

Data Validation Requirements:

1. **On Read:** Always validate schema using Zod before using data
2. **On Write:** Validate data before saving to localStorage
3. **Migration:** Handle schema changes gracefully with version numbers
4. **Corruption Recovery:** Fallback to mock data if localStorage is corrupted

Conflict Resolution:

- **Last Write Wins:** For most data modifications
- **Merge Strategy:** For arrays (append new items, update existing by ID)
- **Optimistic Updates:** Update UI immediately, revert on conflict

2.3 Cross-Component Dependencies

Patient Status Flow:

Check-in Kiosk → waiting_for_front_office
Front Office → waiting_for_ma
MA Dashboard → roomed
MA Intake → ready_for_doctor (+ timestamp)
Doctor Chart → attention_needed (if needed)

Required State Synchronization:

1. **Patient Status Changes:** Must update across all 5 components within 100ms
2. **Lab Result Updates:** Must sync between Technician, Doctor, and Patient portals

- 3. **Claim Status:** Must sync between Doctor Chart and Billing Portal
- 4. **Appointment Changes:** Must sync between Scheduling and Patient Portal

3. Human-in-the-Loop (HITL) Workflow Specifications

3.1 Standard HITL Pattern

Three-Stage Process:

- 1. **AI Processing Stage:**
 - Show loading indicator with specific messages
 - Disable all user inputs
 - Provide cancel option after 15 seconds
- 2. **Human Review Stage:**
 - Present AI results in clearly marked, editable fields
 - Highlight AI-generated content with subtle background color
 - Show "AI-Generated" badges next to suggested items
 - Enable all editing capabilities
- 3. **Human Approval Stage:**
 - Require explicit approval action ("Accept AI Suggestion" or "Approve Changes")
 - Show confirmation dialog for high-impact actions
 - Log the human decision for training data

3.2 Button State Logic

Standard Button States:

typescript

```

interface HITLButtonState {
  aiProcessing: boolean;    // AI is running
  aiResultsReady: boolean;  // AI completed, results available
  humanModified: boolean;   // Human made changes
  approved: boolean;        // Human explicitly approved
}

// Button enable/disable logic:
const canAcceptAI = aiResultsReady && !humanModified && !approved;
const canApprove = (aiResultsReady || humanModified) && !approved;
const canSave = approved;

```

Specific Component Rules:

Billing Portal Review:

- "Ask AI to Fix" button: Enabled only when `flaggedIssues.length > 0`
- "Save Changes" button: Enabled only when `flaggedIssues.length === 0`
- Manual edit triggers: Clear `flaggedIssues` array automatically

MA Patient Intake:

- "Generate Summary" button: Enabled only when `transcript.length > 50`
- "Mark Ready for Doctor" button: Enabled only when `summary` exists and `summary.length > 10`

Lab Result Review:

- "Accept AI Suggestion" button: Enabled only when `aiNotes` exists and not already accepted
- "Approve & Send to EMR" button: Enabled only after AI accepted OR manual notes entered
- "Send Back to Technician" button: Always enabled, opens separate workflow

3.3 Data Capture Requirements

Training Data Collection:

typescript

```
interface HITLInteraction {
  workflowType: 'claim_review' | 'lab_processing' | 'ma_summary' | 'note_generation';
  timestamp: string;
  aiSuggestion: any; // Original AI output
  humanModification: any; // Final human-approved version
  acceptanceRate: boolean; // Did human accept AI suggestion as-is?
  modificationDetails?: { // What specifically was changed
    fieldsChanged: string[];
    changeType: 'correction' | 'addition' | 'removal';
  };
}
```

4. Error States & User Feedback

4.1 Standard Error UI Patterns

Error Display Components:

1. **Inline Errors:** For form validation
2. **Alert Banners:** For workflow interruptions
3. **Modal Dialogs:** For critical errors requiring user action
4. **Toast Notifications:** For background process updates

Error Message Templates:

```
typescript
const ERROR_MESSAGES = {
  AI_TIMEOUT: "AI processing is taking longer than expected. You can continue manually or wait a bit longer.",
  INVALID_FILE: "Please upload a clear image in JPG, PNG, or PDF format.",
  NETWORK_ERROR: "Connection issue detected. Please check your internet and try again.",
  DATA_CORRUPTION: "Data appears corrupted. Refreshing with latest information.",
  VALIDATION_FAILED: "Please check the highlighted fields and try again.",
  PERMISSION_DENIED: "You don't have permission to perform this action."
};
```

4.2 Specific Error Scenarios

File Upload Errors:

- **File too large:** "File size must be under 10MB. Please compress or resize the image."
- **Invalid format:** "Only JPG, PNG, and PDF files are supported."

- **Corrupted file:** "Unable to read file. Please try uploading again."
- **No file selected:** Disable upload button, show helper text

AI Processing Errors:

- **Model unavailable:** Switch to manual mode immediately, show notification
- **Rate limiting:** Show countdown timer, auto-retry when ready
- **Invalid response:** Log error, show "Please try again" message
- **Partial failure:** Show which parts succeeded/failed, allow retry of failed parts

Validation Errors:

- **Required fields:** Red border + specific message below field
- **Format errors:** "Please enter a valid [format]" with example
- **Business logic errors:** "This combination is not allowed because..."

5. Performance & UX Requirements

5.1 Loading Performance Standards

Page Load Requirements:

- **Initial page render:** <500ms
- **LocalStorage data load:** <100ms
- **AI workflow initiation:** <200ms
- **Component state updates:** <50ms

AI Processing Expectations:

- **Document OCR:** 3-8 seconds
- **Note generation:** 5-12 seconds
- **Claim analysis:** 2-5 seconds
- **Simple extractions:** 1-3 seconds

5.2 Responsive Design Breakpoints

Required Breakpoints:

- **Mobile:** 320px - 768px
- **Tablet:** 768px - 1024px

- **Desktop:** 1024px+

Component-Specific Requirements:

- **Tables:** Must scroll horizontally on mobile, show 2-3 key columns
- **Dialogs:** Must be full-screen on mobile, centered on desktop
- **Forms:** Single column on mobile, multi-column on desktop
- **Charts:** Must be readable and interactive on all screen sizes

6. Security & Compliance Requirements

6.1 PHI Handling Rules

Data Classification:

- **PHI Fields:** Patient name, DOB, SSN, medical record numbers, diagnosis codes
- **Non-PHI Fields:** Timestamps, status flags, system-generated IDs
- **Sensitive Fields:** Insurance information, billing amounts

Security Requirements:

1. **Encryption:** All PHI must be encrypted in localStorage using Web Crypto API
2. **Session Management:** Clear all localStorage on browser close
3. **Access Logging:** Log all PHI access attempts (read/write/delete)
4. **Data Minimization:** Only store necessary PHI, purge after session

6.2 Audit Trail Requirements

Required Logging:

```
typescript

interface AuditEvent {
  timestamp: string;
  userId: string;
  action: 'create' | 'read' | 'update' | 'delete' | 'approve' | 'reject';
  resource: 'patient' | 'claim' | 'lab_result' | 'appointment';
  resourceId: string;
  changes?: Record<string, { from: any; to: any }>;
  allInvolved: boolean;
  ipAddress?: string;
}
```

7. Testing Requirements

7.1 Unit Test Coverage

Required Test Categories:

1. **Component Rendering:** All components render without errors
2. **AI Workflow States:** All HITL workflow state transitions
3. **Data Persistence:** LocalStorage read/write operations
4. **Error Handling:** All error scenarios and recovery
5. **Cross-Component Sync:** Storage event handling

Specific Test Cases:

- AI timeout handling in each workflow
- localStorage corruption recovery
- Invalid file upload handling
- Network failure scenarios
- Race condition handling

7.2 Integration Test Scenarios

End-to-End Workflows:

1. **Complete Patient Journey:** Check-in → MA → Doctor → Billing
2. **Lab Result Processing:** Upload → Technician → Doctor → Patient
3. **Cross-Portal Synchronization:** Changes in one portal reflect in others
4. **Error Recovery:** System behavior when AI services fail

8. Development Guidelines

8.1 Code Quality Standards

Required Patterns:

- TypeScript for all components
- Zod schemas for all data models
- Error boundaries around AI workflows
- Loading states for all async operations

- Consistent naming conventions

File Structure:

```
src/  
├── components/  
│   ├── ui/          # Reusable UI components  
│   ├── forms/       # Form components  
│   └── workflows/    # HITL workflow components  
├── lib/  
│   ├── ai/          # AI flow interfaces  
│   ├── data/         # Mock data and schemas  
│   └── utils/        # Utility functions  
├── types/           # TypeScript type definitions  
└── apps/            # Portal-specific pages
```

8.2 AI Integration Patterns

Standard AI Flow Integration:

```
typescript  
  
const useAIWorkflow = <T>(flowName: string) => {  
  const [isProcessing, setIsProcessing] = useState(false);  
  const [result, setResult] = useState<T | null>(null);  
  const [error, setError] = useState<AIFlowError | null>(null);  
  
  const execute = async (input: any) => {  
    setIsProcessing(true);  
    setError(null);  
  
    try {  
      const result = await callAIFlow(flowName, input);  
      setResult(result);  
    } catch (err) {  
      setError(mapToAIFlowError(err));  
    } finally {  
      setIsProcessing(false);  
    }  
  };  
  
  return { execute, isProcessing, result, error, reset: () => setResult(null) };  
};
```

This detailed specification should significantly reduce the back-and-forth with your development team by providing clear, unambiguous requirements for every major aspect of the system.